

Project 4: Fashion-MNIST Data

Isabella Iniguez

2025-12-14

Introduction

The main goal of this project is to do multi-class classification to predict the class label in the Kaggle Fashion MNIST testing data.

The dataset can be downloaded from the following link: [Kaggle Link](#)

0. Data Pre-Processing

Given the size of the data sets, I performed PCA to reduce dimensionality. Arbitrarily, I choose to use the top 100 components.

Given the limitations of my laptop's power, I would also need to take a sample of the training data in order to further reduce dimensionality. To maintain the even representation of each label (as will be shown in Part 1), I used stratified sampling.

Finally, I scaled both the training and test data, as the coming uses of the data are sensitive to scaling.

Now that the data can be more easily processed, we can continue.

1. Data Processing and Unsupervised Learning

(a) Compute and report a frequency table of the outcome variable in both training and testing data.

Training Frequency Table:

##	Count	Percentage
## 0	6000	10
## 1	6000	10
## 2	6000	10
## 3	6000	10
## 4	6000	10
## 5	6000	10
## 6	6000	10
## 7	6000	10
## 8	6000	10
## 9	6000	10

Test Frequency Table:

##	Count	Percentage
## 0	1000	10
## 1	1000	10
## 2	1000	10
## 3	1000	10
## 4	1000	10

```
## 5 1000      10
## 6 1000      10
## 7 1000      10
## 8 1000      10
## 9 1000      10
```

We see that both the original train and test data sets have an equal distribution of each label, taking up 10% of the observations. This means that we have ideal, equal representation of each label when it comes to training and testing our model. Though we will proceed with a portion of the training data, we maintained this distribution in the stratified sampling to adequately represent the original data set.

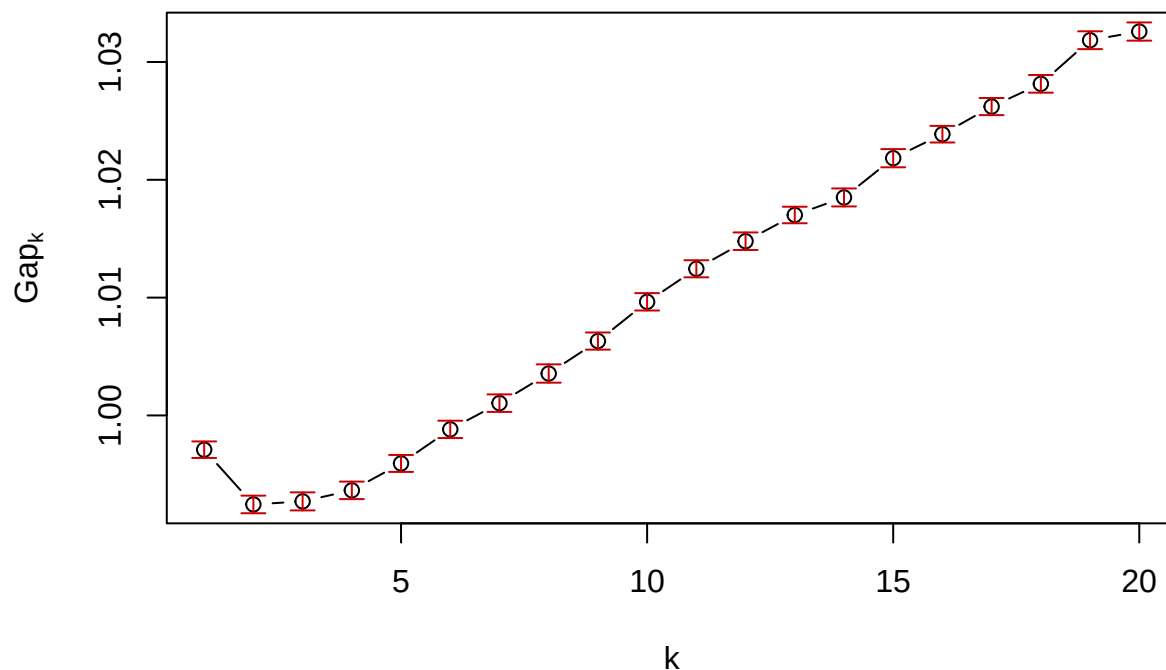
(b) Perform two clustering algorithms to the training data. Comment on the resulting clusters.

The two clustering algorithms I'll be using is K-Means Clustering and Hierarchical Clustering.

First, I'll find the Gap Statistic to determine the optimal number of clusters for the two algorithms to use.

```
## Clustering k = 1,2,..., K.max (= 20): .. done
## Bootstrapping, b = 1,2,..., B (= 20) [one "." per sample]:
## ..... 20
```

Gap Statistic Plot



Since the first instance where the Kth gap statistic exceeds that of K+1 is k=1, we will instead look to the alternate definition of the optimal gap statistic: the highest gap statistic. This is because using k=1 would result in the clustering algorithms simply creating a singular cluster with all the data points, which won't really provide any valuable information regarding dominating labels since the label distribution would just be even.

The resulting Gap Statistic would then be 20.

We can then train the two clustering algorithms with this k_{gap}.

1. K-Means Clustering

The dominating labels in each of the K-means clusters is as shown.

##	Dominating_Label	Count
## 1	2	92
## 2	3	338
## 3	2	177
## 4	5	151
## 5	7	243
## 6	9	224
## 7	4	159
## 8	5	139
## 9	8	184
## 10	0	312
## 11	8	146
## 12	5	120
## 13	2	102
## 14	1	84
## 15	9	89
## 16	8	156
## 17	1	223
## 18	9	207
## 19	6	265
## 20	1	198

The K-Means clusters do somewhat help separate the labels since for each label there is at least one cluster where it is the dominating label.

2. Hierarchical Clustering:

The dominating labels in each of the hierarchical clusters is as shown.

##	Dominating_Label	Count
## 1	8	223
## 2	3	461
## 3	5	63
## 4	8	24
## 5	9	40
## 6	1	23
## 7	9	26
## 8	8	47
## 9	8	13
## 10	0	1
## 11	8	5
## 12	2	1
## 13	5	34
## 14	9	17
## 15	8	3
## 16	5	3
## 17	6	1
## 18	8	1
## 19	8	1
## 20	8	1

The hierarchical clusters don't help separate the labels since not every label is dominating at least one cluster. Labels 4 and 7 (representing Coats and Sneakers) don't dominate any cluster.

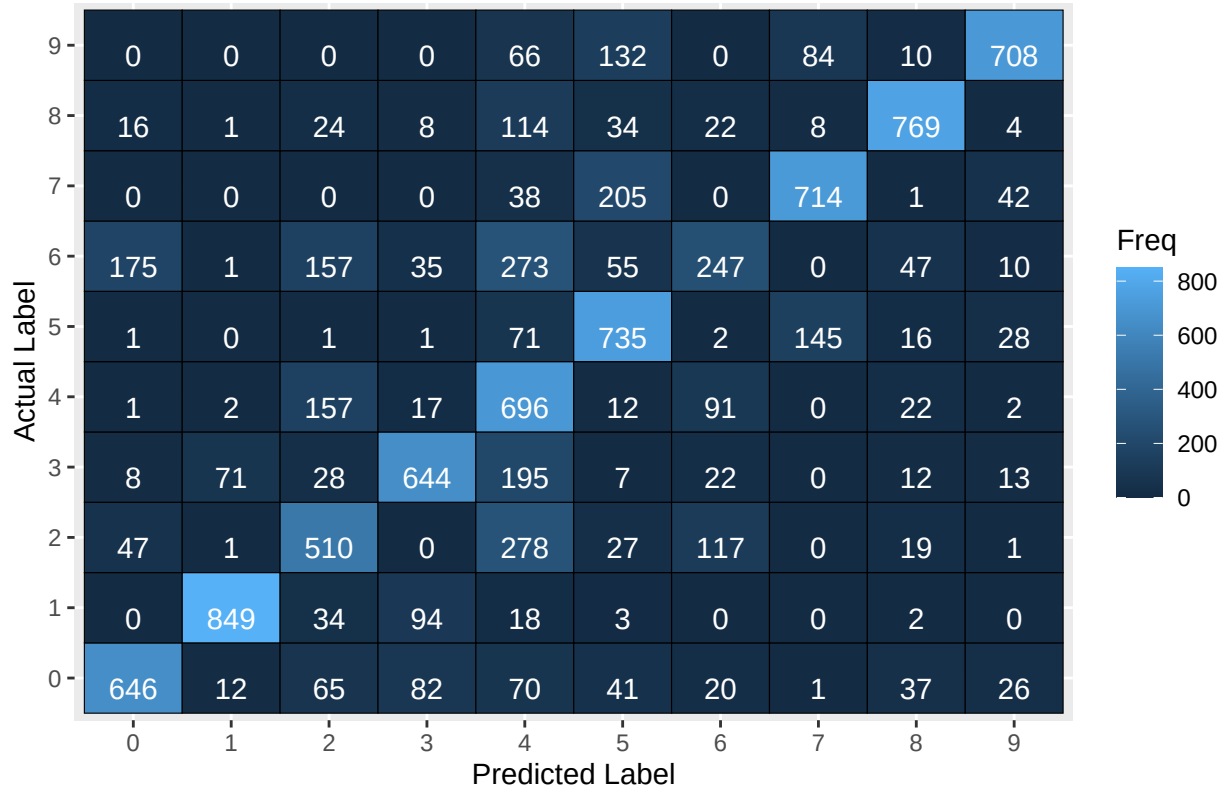
2. Multi-class Classification Model

Here, I used four of the methods we discussed in class to do multi-class classification: Decision Tree, Random forest, GBM, and XGBoost Models.

1. Decision Tree

I decided to start with a pruned decision tree model. I primarily chose to prune to help reduce overfitting. We are working with a fraction of the available training data, thus I am concerned with the model struggling to generalize to a wider variety of data. The rest of the parameters were left to their defaults.

Decision Tree Confusion Matrix

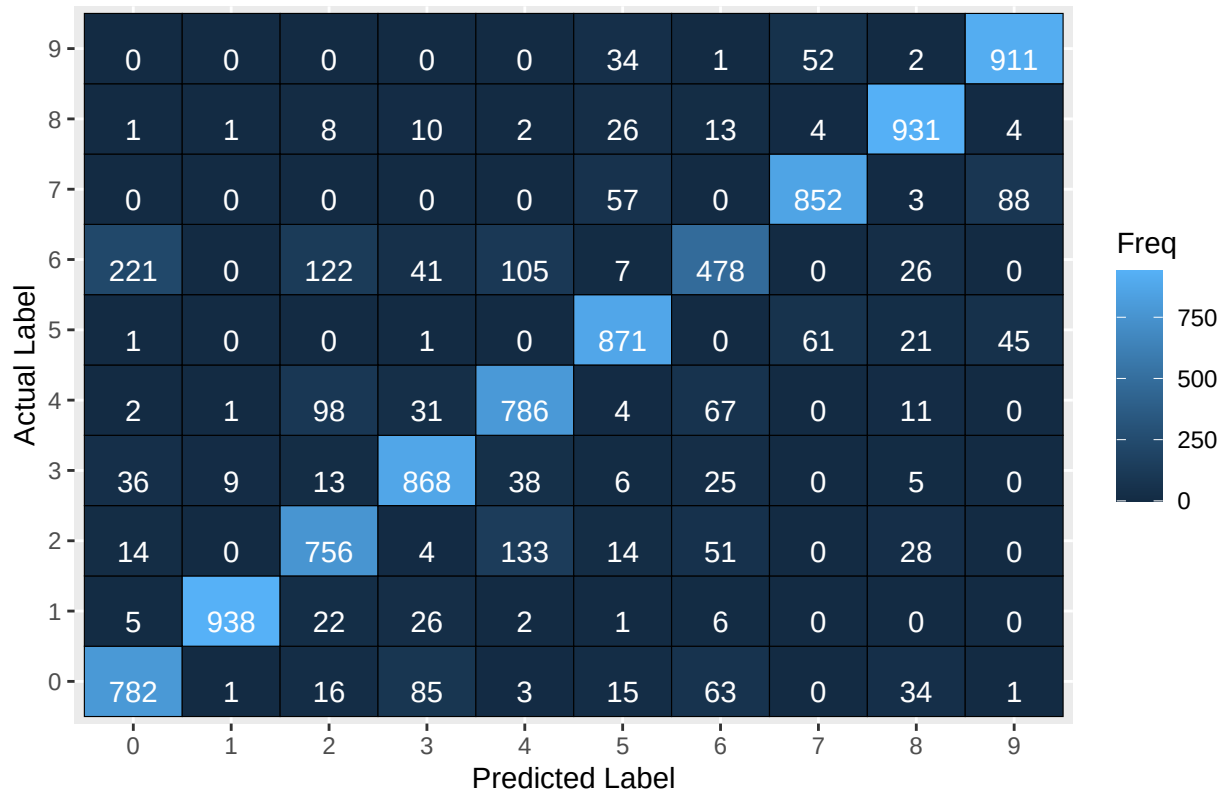


The tree model resulted in a classification error of 0.3482. From the heatmap, it appears as if it struggles to predict Shirts (6) the most, misclassifying shirts as Coats (4), Pullovers (2), or T-shirt/tops (0). This isn't surprising since we know shirts to be visually similar to these other items, thus they are likely to be confused for one of them given the limitations of a single tree. Otherwise, the model does perform fairly well.

2. Random Forest

To improve from a single decision tree, I decided to try a random forest model. I choose to use a safe 1000 decision trees. Wanting to mitigate the computation time, I used conservative estimates for the rest of the parameters, possibly reducing the overall model's predictive power ($mtry = 7$, $nodesize = 10$, $samplesize = 800$). However, compared to the following models trained in this project, the model's performance is fairly consistent.

Random Forest Confusion Matrix

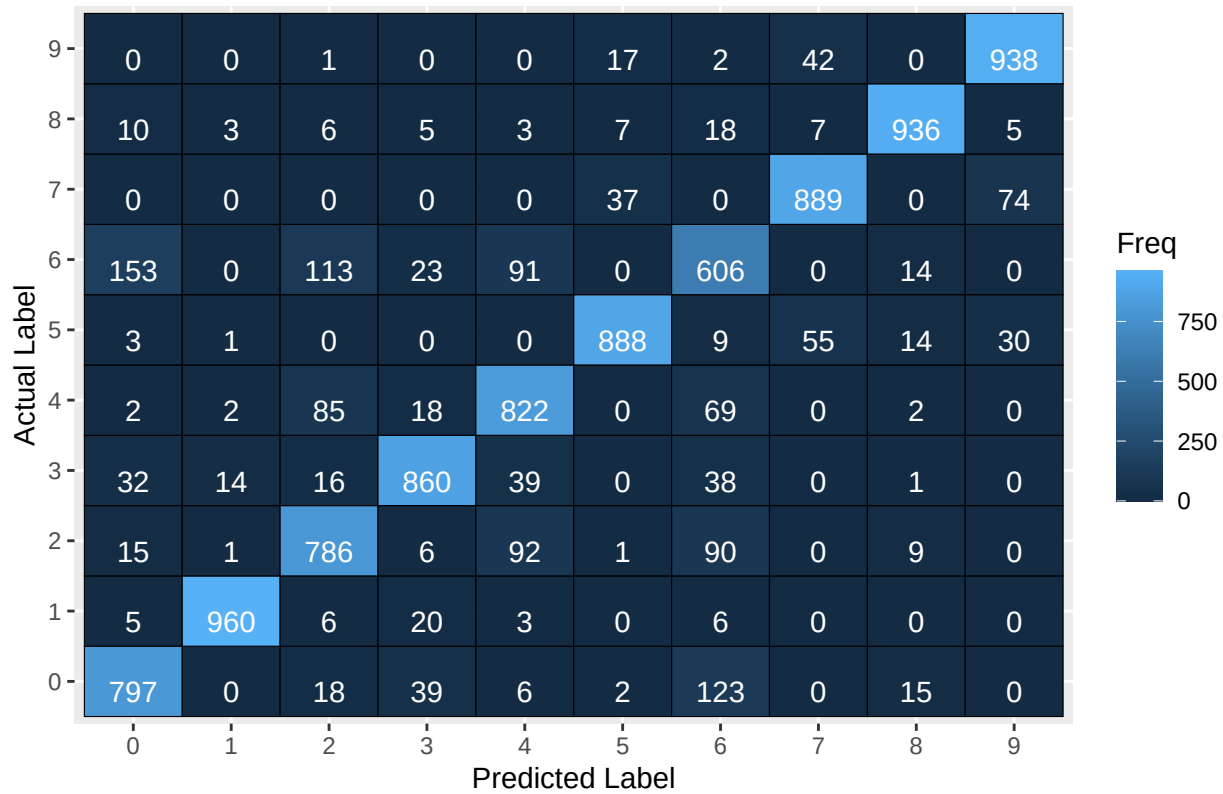


The random forest model resulted in a classification error of 0.1827. As with the random tree model, it also appears to struggle the most with predicting Shirts (6). As expected, the random forest performs better than the single tree model.

3. GBM

Moving onto a GBM model, I kept the 1000 decision trees but set the remaining parameters to their default values and adjusted by eye. I wanted a fair interaction depth given we are analyzing images, so I choose 5.

GBM Confusion Matrix

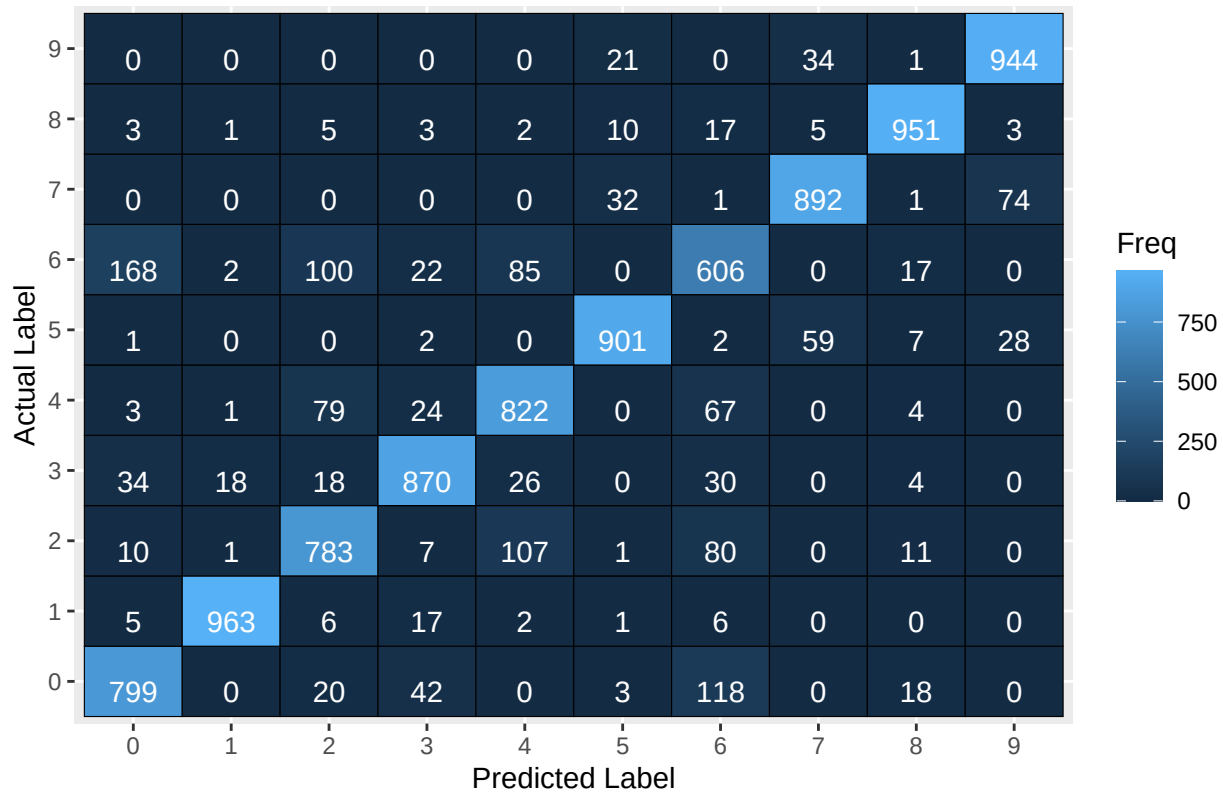


The GBM model resulted in a classification error of 0.1518. As with the previous models, it also struggles the most with predicting Shirts (6).

4. XGBoost

Then finally the XGBoost model, I similarly tuned the parameters by eye.

XGBoost Confusion Matrix



The XGBoost model resulted in a classification error of 0.1469. As with the previous models, it also struggles the most with predicting Shirts (6), though it appears to have performed the best overall.

Report the overall classification error.

The overall classification error is 0.2074.